

AggreGuard: Trace-Level, Aggregation-Aware Guardrails for LLM Agents

*Workshop short-paper draft. The three arXiv references are verified (2026-06-26); author

list and venue formatting remain to be finalized before submission. Named tools and

standards are cited as software/products rather than papers.*

Abstract

LLM agents now take consequential actions through tools, and most guardrails defend them one message or one action at a time. We observe that this leaves a class of privacy attacks unaddressed: a sequence of individually-benign disclosures that, in aggregate, either crosses a sensitivity threshold or re-identifies an individual by combining quasi-identifiers—the inference-and-aggregation problem from statistical databases, reappearing on the agent's execution trace. We present AggreGuard, a trace-level guardrail middleware whose distinctive component is an aggregation-aware monitor that scores cumulative disclosure per data subject and escalates on either a weighted sensitivity budget or a k-anonymity re-identification check against a reference population, preferring human-in-the-loop escalation to hard blocking. Because no existing agent benchmark exercises multi-step aggregation, we build a reproducible attack suite and an evaluation harness, and we sweep the monitor's parameters to trace a detection–false-positive frontier. Against a fair stateful field-counting baseline that matches the monitor on raw detection, the monitor attains a markedly better trade-off—0.989 versus 0.663 detection at a 2% false-positive budget on the primary run, with the gap stable across seeds—because it reasons about re-identifiability rather than counting fields. We report this against an independent privacy standard with an incomplete reference sample, and we are explicit about the demonstration's limits.

1. Introduction

Large language model (LLM) agents increasingly act on the world rather than merely converse with it: they read email, browse the web, query databases, and call tools that

send messages, move money, or change records. This shift turns the model's outputs into consequential actions and correspondingly widens the attack surface. The defining threat in this setting is indirect prompt injection, in which instructions embedded in untrusted content the agent retrieves—a web page, a document, a tool result—hijack the agent into acting against the user's interest. The community has responded with dedicated benchmarks that quantify how often such attacks succeed and how well defenses hold, most prominently AgentDojo, which pairs realistic tasks with injection cases across tool-using environments [1].

Most deployed guardrails, however, reason locally. Input/output text filters scan each message or tool result in isolation for injection patterns or disallowed content; architectural defenses constrain how a single untrusted value may flow into a privileged call [2]; and task-alignment monitors check whether each proposed action serves the user's stated intent [3]. These mechanisms are valuable and largely complementary, but they share a blind spot: none maintains a session-level account of *how much* an agent has cumulatively disclosed about a given data subject. Consequently, a class of attacks slips through in which no single step is suspicious. An agent can answer a series of individually reasonable questions—one attribute at a time—until the aggregate either exceeds any reasonable sensitivity threshold or, by combining innocuous quasi-identifiers such as a postal code, a gender, and a birth year, uniquely re-identifies an individual. This is precisely the inference-and-aggregation problem long studied for statistical databases [4, 5, 6], now reappearing on the agent's execution trace. Stateless, per-step filters cannot see it because each step, viewed alone, is benign.

We present AggreGuard, a trace-level guardrail middleware for LLM agents whose distinctive component is an aggregation-aware monitor. AggreGuard composes a small set of defenses—data provenance and trust-boundary enforcement, an off-the-shelf injection detector, a task-alignment check, high-risk action gating, decision logging, and the aggregation

monitor—into a single per-step decision that an integration can mount on an agent framework. The aggregation monitor is the novel element: it tracks cumulative disclosure per data subject across the trace and escalates when a weighted sensitivity budget is exceeded or when the subject's anonymity set, estimated against a reference population, falls below a threshold. To keep the monitor usable, it prefers human-in-the-loop escalation over hard blocking, scores only disclosures that cross the trust boundary, and scopes risk to individual subjects so that ordinary multi-step work does not accumulate. We make three contributions. (1) We adapt classical inference-control ideas—a cumulative sensitivity budget and k-anonymity over a reference population—into a runtime monitor that operates on an agent's tool-call trace rather than on database queries. (2) We build a reproducible, zero-cost aggregation-inference attack suite together with an evaluation harness that compares the monitor against both a stateless injection filter and a fair stateful baseline. (3) We report a detection–false-positive trade-off, swept over the monitor's parameters, showing that reasoning about re-identifiability dominates a field-counting baseline: at a 2% false-positive budget the monitor detects 0.989 of aggregation attacks versus 0.663 for the field-counter, a gap that is stable across seeds. We are deliberate about scope: this is a mechanism demonstration on a synthetic suite, and we report the result against an independent privacy standard with an incomplete reference sample so that the monitor is not an oracle.

2. Related Work

Guardrails and prompt-injection defenses. A growing ecosystem of guardrails inspects the text entering or leaving an LLM. Open-source toolkits such as LLM Guard [7] and dialogue flow systems such as NeMo Guardrails [8] chain scanners and policies, and commercial services such as Lakera Guard [9] offer low-latency input/output scanning; safety classifiers such as Llama Guard and ShieldGemma [10], and dedicated injection detectors such as ProtectAI's DeBERTa-based prompt-injection classifier and Meta's

Prompt-Guard [11], provide per-message judgments. These operate at the input/output text boundary and treat each message independently. A second line moves the defense into the agent's architecture: CaMeL isolates untrusted data so it cannot directly parameterize privileged actions [2], while task-alignment approaches such as Task Shield judge each action against the user's original intent [3]. A third, recent line analyzes the agent's runtime trace with program-analysis techniques to detect injection. AggreGuard is complementary to all three: it is trace-level and composable like the architectural and trace-analysis defenses, it reuses an off-the-shelf injection detector for the single-step case rather than reinventing it, and its distinguishing capability—cumulative, subject-scoped disclosure accounting—is absent from every one of them.

Benchmarks for agent security. Evaluating these defenses requires environments that combine realistic tasks with attacks. AgentDojo is the de facto standard, providing tool-using suites with paired user tasks and injection tasks and a harness for measuring attack success and utility under defense [1]; complementary efforts contribute additional injection scenarios and a notion of attack concealment. Our injection-side experiments use AgentDojo directly. Critically, however, these benchmarks target single-step injection; none exercises multi-step aggregation, which is why we contribute our own suite for the aggregation monitor.

Inference control in statistical databases. The theoretical roots of the aggregation monitor lie in decades of work on protecting privacy when releasing data. k -anonymity formalizes re-identification risk through the size of the anonymity set induced by a set of quasi-identifiers [4, 5]; the query-auditing literature studies when a sequence of individually permissible queries must be refused because their combination discloses too much; and differential privacy frames cumulative disclosure as a budget that depletes with each release [6]. AggreGuard ports these ideas from the database query interface to the LLM agent's tool-call trace: the sensitivity budget is the agent-trace analogue of a privacy

budget, and the anonymity-set check is k -anonymity applied to the quasi-identifiers an agent has emitted about a subject. To our knowledge, no existing LLM-agent guardrail incorporates this aggregation-aware perspective, which is the gap this work addresses.

3. Method

The aggregation monitor maintains a disclosure ledger keyed by the pair (session, data subject). A disclosure is recorded only when data linkable to a subject flows to an *untrusted* sink, that is, an action that leaves the trust boundary such as sending an email, posting externally, or writing to an attacker-controllable channel; reads to internal sinks are not scored. This sink gating is essential to keeping false positives low, because it allows an agent to process a single subject in depth internally without ever accumulating risk. Disclosures are further scoped to individual subjects, so a bulk-but-shallow workflow that touches many subjects with one harmless field each never concentrates risk on any one of them.

The monitor applies two complementary escalation criteria. The first is a cumulative sensitivity budget. Each attribute a carries a weight $w(a)$ reflecting how sensitive it is, and the monitor tracks the running sum $S(\text{subject}) = \sum w(a)$ over the distinct facts disclosed externally about that subject; the same fact disclosed twice is counted once. When S reaches a threshold τ , the monitor escalates. This budget formalizes the "death by a thousand cuts" intuition in the spirit of a privacy budget [6], and by default the budget does not decay over time, because a fact, once leaked, remains leaked—decaying the budget would let a patient adversary defeat the mechanism simply by spacing disclosures out. The second criterion targets re-identification directly. The monitor maintains the set of quasi-identifiers disclosed about a subject and estimates the subject's anonymity set against a reference population, namely the number of population members consistent with all disclosed quasi-identifier values. Following the k -anonymity principle [4, 5], it escalates when the anonymity set falls below k . Crucially, an anonymity set of zero is treated as

"the subject is not a member of the reference population"—the least re-identifiable case rather than the most—so the monitor escalates only when the set is small but non-empty ($0 < \text{anon} < k$).

When either criterion fires, the monitor raises a human-in-the-loop (HITL) escalation rather than a hard block. Because re-identifiability and cumulative sensitivity are properties of the data flow and not of the user's intent, a legitimate task may legitimately need to disclose identifying information to an authorized party; converting such cases into a single confirmation prompt, rather than an outright denial, keeps the mechanism usable while preserving its protective value. The thresholds τ and k , the attribute weights $w(a)$, and the quasi-identifier set are the monitor's tunable parameters and define its operating point.

4. Evaluation

4.1 Setup

Because no existing agent benchmark exercises multi-step aggregation, we constructed an aggregation-inference attack suite. Each scenario is a sequence of disclosure events, labelled as either harmful or benign, in which every individual step is innocuous. The harmful scenarios comprise slow sensitivity leaks (e.g., disclosing a subject's salary and then their medical condition to an external sink) and quasi-identifier re-identifications (disclosing a zip code, gender, and birth year whose combination is unique in the population). The benign scenarios are deliberately chosen to stress false positives: deep multi-field processing routed only to an internal sink, single-attribute disclosures, bulk shipping updates spanning many subjects, and—most importantly—legitimate disclosures of multiple quasi-identifiers about one subject whose combination does *not* re-identify them. We compare the monitor against two baselines. The first is a stateless prompt-injection text filter, included to show that an injection detector is simply the wrong tool for this threat: it inspects one message at a time for override phrasing and has no notion of

cumulative PII disclosure. The second is a deliberately fair, non-novel baseline: a stateful, egress-mounted PII field-counter that, per subject, counts the distinct sensitive fields disclosed externally and escalates once that count reaches a threshold. This baseline captures the obvious "just add session memory" objection to the aggregation monitor, and it matches the monitor on raw detection; the question the evaluation answers is whether the monitor's principled scoring yields a better detection/false-positive trade-off than counting fields.

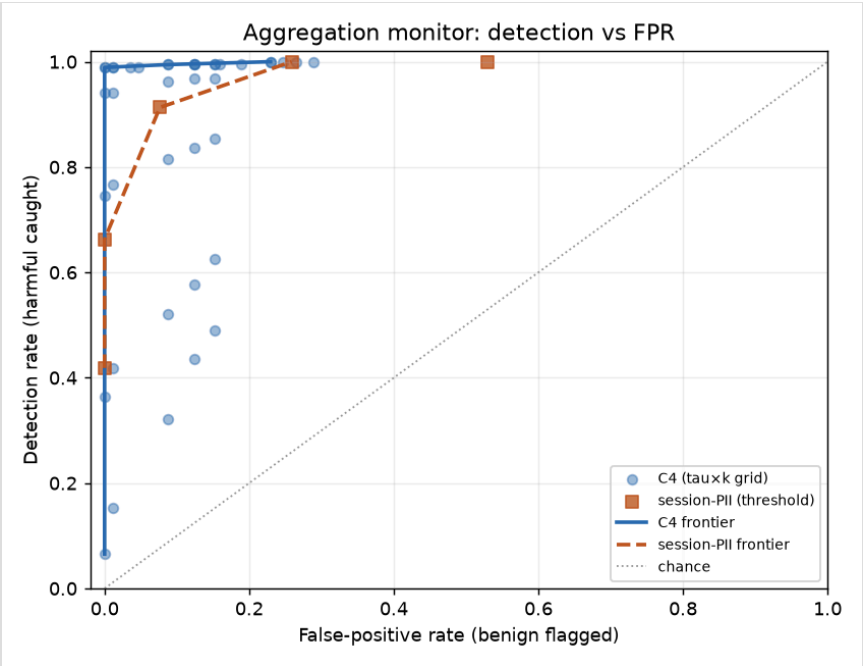
To measure that trade-off without a single hand-tuned operating point, we ran a parameter sweep over a grid of thresholds (τ, k) for the monitor and over its threshold for the field-counter, on a larger generated benchmark of 354 labelled scenarios (184 harmful, 170 benign) drawn from a 371-member synthetic population with varied cell multiplicities so that anonymity-set sizes span the full range from unique to common. Three measures guard against an over-favorable result. First, ground-truth harm is defined by an *independent* privacy standard—its own weight table and thresholds, distinct from those the monitor uses—and ambiguous scenarios are dropped to avoid label noise. Second, the monitor estimates anonymity from an *incomplete* reference sample comprising roughly half the population, mirroring the fact that real k-anonymity is computed against an external reference rather than the true population, so the monitor is not an oracle on the re-identification axis. Third, the monitor's attribute weights differ from the ground-truth weights, so it is not an oracle on the budget axis either.

4.2 Results

On the core suite under the system's shipped configuration, the prompt-injection filter detected none of the aggregation attacks, confirming that it is the wrong instrument for this threat. The field-counter and the aggregation monitor both detected every attack, but they differed sharply on false positives: the field-counter flagged 40% of the benign scenarios, whereas the monitor flagged none. The two scenarios that separate them are

exactly the legitimate multi-quasi-identifier disclosures—cases in which several identifying fields are shared for a benign reason but their combination does not re-identify the subject. The field-counter, which only counts fields, cannot distinguish these from an attack; the monitor, which reasons about the resulting anonymity set, correctly allows them.

The parameter sweep generalizes this finding into a detection–false-positive trade-off curve (Figure 1). Across the grid, the monitor's achievable frontier lies above and to the left of the field-counter's at every operating point. At a tight false-positive budget of 2%, the monitor attains a detection rate of 0.989 against the field-counter's 0.663; the field-counter must be loosened to roughly a 25% false-positive rate before it matches the monitor's detection. Because an exact-zero-false-positive operating point is a brittle knife-edge under an incomplete reference, we report detection at small false-positive budgets rather than at exactly zero, and we verified that the advantage is stable: across eight population-by-scenario seed combinations, the monitor's detection at a 2% budget remains in the range 0.98–1.00 while the field-counter's stays at 0.66–0.68.



point for the aggregation monitor (circles) or one threshold for the stateful PII

field-counter (squares); solid and dashed lines trace each method's achievable frontier.

The monitor's frontier dominates, achieving higher detection at every false-positive level.](../benchmarks/results/aggregation/tradeoff_curve.png)

Figure 1. Detection versus false-positive trade-off on the 354-scenario benchmark. The aggregation monitor (blue) is swept over (τ, k) ; the stateful PII field-counter (orange) is swept over its count threshold. The monitor's frontier dominates the field-counter's, indicating higher detection at any fixed false-positive rate.

The interpretation is specific and we state it precisely: the result demonstrates that the *right feature space*—weighted cumulative sensitivity together with an anonymity-set estimate—dominates field-counting for this trade-off, not that the monitor detects privacy harm defined by some arbitrary external standard.

To confirm the field-counter is not a straw man for off-the-shelf tools, we also ran the real LLM Guard (Protect AI) package over the same suite. Its prompt-injection scanner detected none of the aggregation attacks, as expected of an injection detector, and its Sensitive scanner—a production PII detector backed by Microsoft Presidio—detected only 0.50 of the attacks at a 0.20 false-positive rate. The Sensitive scanner flags messages that contain overt PII (a medical condition, a salary) but, scanning each message in isolation with no session state, it both misses the quasi-identifier re-identification attacks, whose individual fields are not themselves sensitive, and over-flags benign messages that merely mention PII. This is the same structural limitation the field-counter exhibits, now established with the deployed package rather than a stand-in.

4.3 Limitations

Several limitations bound these claims. The attack suite is small and synthetic; it demonstrates the mechanism and its trade-off but is not a large-scale benchmark, and the synthetic population, while varied, does not capture the heavy-tailed cell-size distributions of real microdata. The re-identification branch matches quasi-identifiers by

canonicalized attribute name, so an adversarial tool that relabels a field with an unrecognized synonym can evade the anonymity-set check; this is a known weakness shared with keyword-based defenses and motivates value-level rather than name-level matching in future work. Finally, the ground-truth privacy standard in the sweep, although independent of the monitor's parameters, is necessarily defined over the same two concepts the monitor reasons about—cumulative sensitivity and anonymity—so the experiment establishes the superiority of these features over field-counting rather than absolute detection of harm under an unrelated definition. We regard calibrating τ and k on held-out benign traffic, and validating against attacks that do not share the monitor's feature space, as the central items for a full evaluation.

4.4 Single-step injection on AgentDojo

To verify that AggreGuard's per-step components run end to end on a real agent framework, and to situate them relative to the single-step injection threat the literature already studies—separate from the aggregation monitor evaluated in §4.2—we ran the system on AgentDojo's workspace suite (v1.2.1) under the `important_instructions` indirect-injection attack [1]. We evaluated two backends: a local `llama3.1:8b` model over 40 user tasks crossed with 3 injection tasks (120 attacked runs); and the hosted `claude-sonnet-4-6` over a smaller 8x3 grid (24 runs). In both settings the baseline attack success rate (ASR) with no defense mounted was exactly zero, but for opposite reasons. The local model is too weak to be hijacked because it is too weak to complete the tasks at all: its clean utility is only 0.025, so there is essentially no capability for an injection to redirect. The hosted model sits at the other extreme—clean utility 1.0, utility-under-attack 1.0, and ASR 0.0—because it is strong and safety-aligned enough to recognize and refuse the injected instructions on its own. The immediate consequence is a null result for end-to-end defense: with baseline ASR already at zero on both backends, there is no headroom for AggreGuard or any other defense to demonstrate an ASR reduction. This is not evidence of the defense working, only that nothing was left for it to do. The samples are also small enough that we cannot claim the true ASR is zero; for the 24-attack hosted grid, a 95% Wilson interval places the upper bound on the true ASR at roughly 14%.

This null result is what the paper's actual claim predicts. The contribution of AggreGuard is the aggregation monitor, which is orthogonal to single-step injection: it addresses cumulative, multi-step disclosure that no single message reveals, a threat AgentDojo does not exercise. On single-step injection, AggreGuard's provenance and trust-boundary enforcement, off-the-shelf injection detector, and task-alignment check are standard components that reuse existing mechanisms rather than improve on them, so with the baseline already at zero there is nothing for them to improve on the two models we tried. What the hosted run does establish is narrower. The `none`-versus-AggreGuard comparison is identical on every metric (clean utility 1.0, ASR 0.0, utility-under-attack 1.0), so AggreGuard does not degrade utility on these traces; we note, however, that the defense did not trigger here—its injection detector fired on 0 of 35 traces (24 attacked plus 11 benign) per arm—so non-degradation is a sanity check on an inert defense rather than a measured utility/safety trade-off. A trace-level audit confirmed the injection payload was present in all 24 attacked traces, so the attacks were genuinely mounted, not dropped.

Beyond confirming that the harness executes end to end on a real API model, the experiment adds an offline detection check computed at zero additional API cost by replaying the cached tool outputs through the injection detector. This measures a different quantity from the end-to-end ASR that the strong model already drives to zero: whether the guardrail would flag the injected content, not whether the attack succeeded. The shipped keyword detector initially fired on 0 of 35 traces because its patterns did not match AgentDojo's actual payload phrasing. After correcting the patterns to match that phrasing, the detector flagged all 24 injected tool outputs—a figure partly overfit, since the patterns were tuned on this attack's phrasing and should not be read as a general detection rate—while flagging none of the 11 benign tool outputs, on both the `none` and `aggreguard` arms. The 0% false-positive rate on benign outputs, untuned against the benign set, is the more trustworthy of the two numbers.

4.5 Component ablation

To attribute the composed guard's behavior to its parts, we ran the full

`AggreGuard.evaluate()` decision over the aggregation suite with subsets of components enabled. The result is unambiguous: enabling only the aggregation monitor (C4) reproduces the full guard exactly—detection 1.00, benign false-positive rate 0.00—while every subset that excludes C4, including each of the other components alone (provenance, the injection detector, task alignment, action gating), detects none of the aggregation attacks. Dropping C4 from the full guard collapses detection to zero. This is the expected and honest reading: the other components target threats this suite does not exercise—high-risk-tool misuse, single-message injection, intent deviation—and are silent here by construction, so the end-to-end aggregation detection is attributable entirely to C4. The ablation's second value is a composition-safety check: adding the other components alongside C4 introduces no false positives on the benign scenarios, so composing the standard defenses around the novel monitor does not degrade its operating point.

5. Conclusion

We presented AggreGuard, a trace-level guardrail middleware whose distinctive component is an aggregation-aware monitor that maintains a per-subject disclosure ledger and escalates on either a cumulative sensitivity budget or a k-anonymity re-identification check against a reference population. The central empirical result is a moat over the obvious "just add session memory" defense: against a fair, stateful PII field-counter that matches the monitor on raw detection, the monitor attains a markedly better detection–false-positive trade-off—0.989 versus 0.663 detection at a 2% false-positive budget on the primary run, with the gap stable across eight population-by-scenario seeds (monitor 0.98–1.00 versus field-counter 0.66–0.68 detection)—because it reasons about re-identifiability rather than counting fields. This advantage holds

even though ground-truth harm is set by an independent privacy standard and the monitor estimates anonymity from an incomplete reference sample, so the monitor is not an oracle on either axis.

We are deliberate about scope. The aggregation result is a mechanism demonstration on a synthetic suite, and it establishes that the right feature space dominates field-counting for this trade-off rather than that the monitor detects privacy harm under some unrelated external definition. The single-step injection experiments on AgentDojo are a null result: baseline ASR is already zero on both backends we tried, for opposite reasons, so no end-to-end ASR reduction is demonstrable there, and the value of those runs is limited to confirming that the harness executes end to end on a real API model, that AggreGuard does not degrade utility on those traces, and that the injection detector flags injected content offline.

Several concrete steps would turn this demonstration into a full evaluation. First, the monitor should be tested against held-out attacks that do not share its feature space, since the current sweep necessarily defines ground-truth harm over the same two concepts—cumulative sensitivity and anonymity—that the monitor reasons about. Second, the re-identification branch should match quasi-identifiers at the value level rather than by canonicalized attribute name, closing the evasion in which an adversarial tool relabels a field with an unrecognized synonym. Third, the thresholds τ and k should be calibrated on real benign traffic rather than synthetic scenarios, and the whole suite should be scaled to larger and more realistic populations whose heavy-tailed cell-size distributions match real microdata. We regard these as the path from a credible mechanism to a deployable guardrail.

References

arXiv entries [1]–[3] verified against arXiv on 2026-06-26 (title, authors, year);

entries without identifiers are software, products, or standards.

[1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr.

AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM

Agents. arXiv:2406.13352, 2024.

[2] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi,

A. Terzis, and F. Tramèr. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813,

2025.

[3] F. Jia, T. Wu, X. Qin, and A. Squicciarini. The Task Shield: Enforcing Task Alignment to

Defend Against Indirect Prompt Injection in LLM Agents. arXiv:2412.16682, 2024.

[4] L. Sweeney. k-Anonymity: A Model for Protecting Privacy. International Journal of

Uncertainty, Fuzziness and Knowledge-Based Systems, 10(5):557–570, 2002.

[5] P. Samarati and L. Sweeney. Protecting Privacy when Disclosing Information:

k-Anonymity and Its Enforcement through Generalization and Suppression. SRI Technical

Report, 1998.

- [6] C. Dwork. Differential Privacy. In Proc. ICALP, 2006.
- [7] Protect AI. LLM Guard: an open-source toolkit for securing LLM interactions (software). <https://github.com/protectai/llm-guard>
- [8] NVIDIA. NeMo Guardrails: a toolkit for adding programmable guardrails to LLM applications (software).
- [9] Lakera. Lakera Guard: prompt-injection and content-safety API (product).
- [10] Meta. Llama Guard; Google. ShieldGemma (LLM safety classifiers).
- [11] Protect AI. deberta-v3-base-prompt-injection-v2; Meta. Prompt-Guard (prompt-injection classifier models).
- [12] OWASP. Top 10 for Large Language Model / Agentic Applications (standard).